

DATE : 16.03.2026

DT/NT : DT

LESSON : GIT & GITHUB

SUBJECT: Git & Github-2

BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com

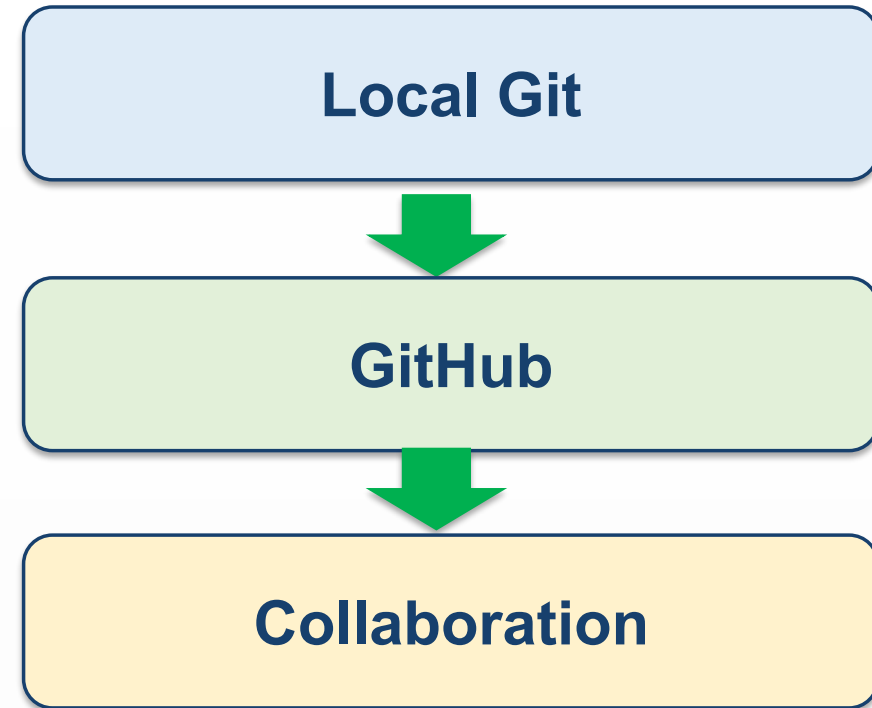


+1 (585) 304 29 59



What are we doing today?

- Remote Repository & GitHub
- Connecting Local Git to GitHub
- Push • Clone • Fetch • Pull
- GitHub Authentication
- .gitignore
- Watch • Star • Fork • Issues
- Feature Branches & Pull Requests
- Code Review • Collaboration • Protection



Day 1: Local → Day 2: Remote + Team

Day 1 — Quick Review



Commands we already know

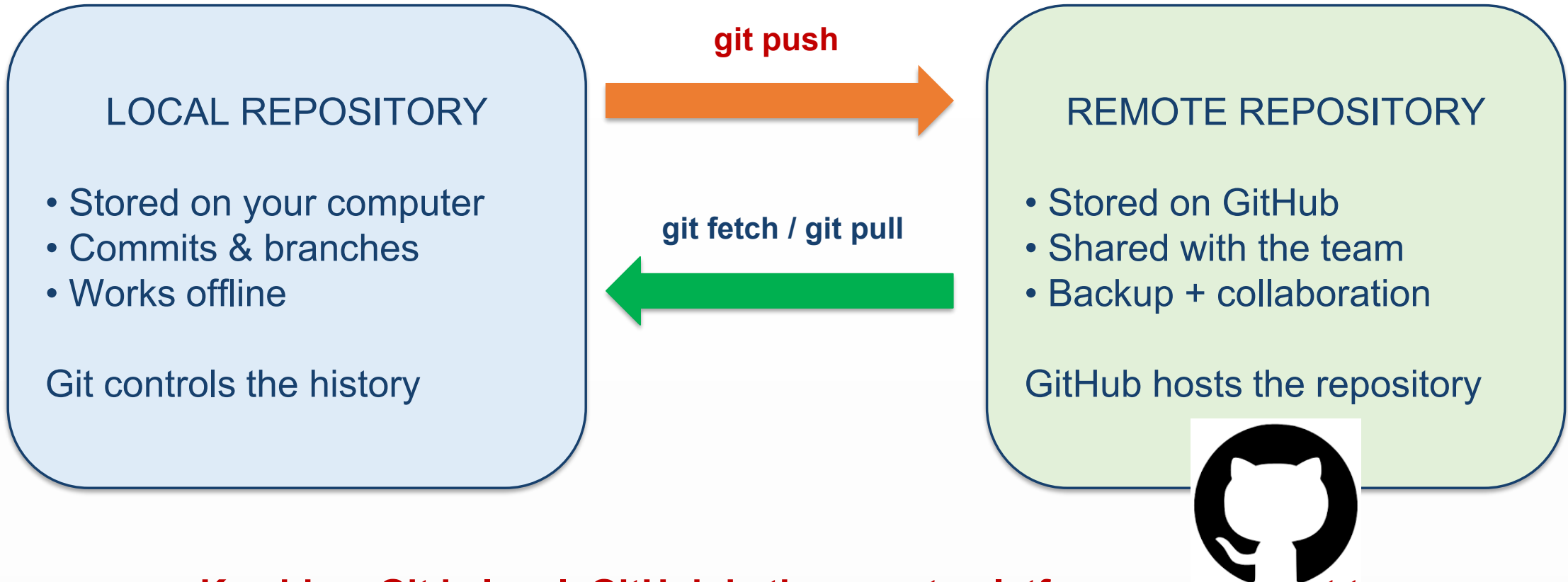
```
git status  
git add <file>  
git commit -m "message"  
git log --oneline
```

Branch workflow

```
git switch -c feature  
git switch main  
git merge feature
```

Today we connect the Local Repository to a Remote Repository on GitHub.

Local Repository vs Remote Repository



Key idea: Git is local. GitHub is the remote platform we connect to.

Creating a GitHub Repository

On GitHub

- New repository
- Choose a simple repository name
- For this example, keep it empty
- Copy the HTTPS or SSH repository URL

Connect an existing local repository

```
git init  
  
git add .  
git commit -m "Initial commit"  
git branch -M main  
git remote add origin <repo-url>  
git push -u origin main
```

Repository = the Git project. GitHub Projects is a separate GitHub planning feature.

Local Repository → origin → GitHub Repository

Remote, origin & git push

origin

“origin” is the conventional name for the remote repository.

```
git remote -v  
git remote add origin <repo-url>
```

First push

```
git push -u origin main
```

- push = send local commits
- origin = remote name
- main = branch name
- -u = remember the upstream branch

Later pushes

```
git push
```

git clone — Copy a Remote Repository

When the project already exists on GitHub:

```
git clone <repository-url>  
cd <repository-name>
```

- Downloads the project files
- Downloads the Git history
- Creates a local Git repository
- Automatically creates origin
- Checks out the default branch

GitHub Repository

git clone



Local Repository

Check the remote after cloning:

```
git remote -v
```

git fetch vs git pull

git fetch

Downloads information about remote changes.

It does NOT automatically integrate those changes into your current branch.

git fetch

git pull

Downloads remote changes AND integrates them into your current branch.

Think: fetch + integrate.

git pull

Use fetch when you want to inspect first.

GitHub Authentication — SSH & HTTPS

SSH

Private key → stays on your machine
Public key → added to GitHub

Good for regular command-line work.

```
ssh-keygen -t ed25519 -C  
"you@example.com"  
cat ~/.ssh/id_ed25519.pub  
ssh -T git@github.com
```

HTTPS

Use a Personal Access Token (PAT)
instead of an account password.

Credential Manager or GitHub CLI can
store authentication securely.

Never share private keys or access tokens.

.gitignore — What should Git ignore?

Common patterns

```
*.log  
.env  
node_modules/  
target/  
*.pem  
*.key
```

If a file is already tracked:

```
git rm --cached <file>
```

DevOps rule

DO NOT COMMIT

- passwords
- API / access tokens
- AWS credentials
- private keys
- secret .env files

Then add it to .gitignore

.gitignore prevents NEW tracking; it does not automatically forget a file already committed.

GitHub Watch | Star | Fork

WATCH

Receive repository notifications.

Useful when you actively follow a project.

STAR

Bookmark / show interest in a repository.

Easy way to find useful projects later.

FORK

Create your own GitHub copy of another repository.

Common for contributing when you do not have direct write access.

For collaboration, Fork is the most important concept of these three.

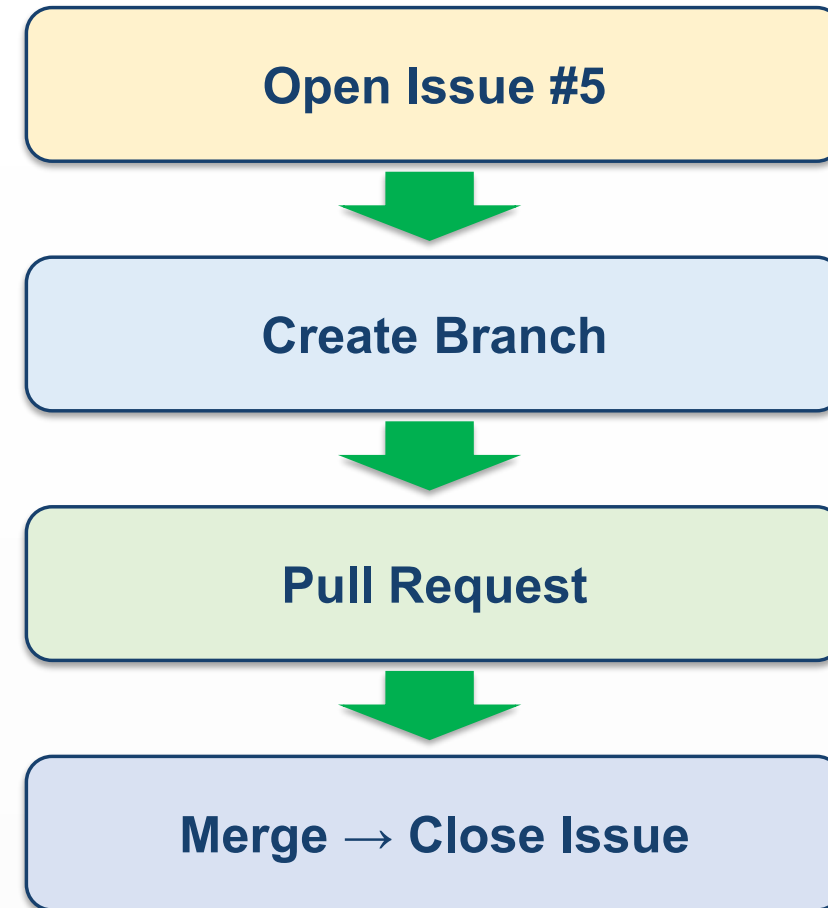
GitHub Issues

Use Issues to track:

- Bugs
- Tasks
- Feature requests
- Questions / suggestions

Helpful fields

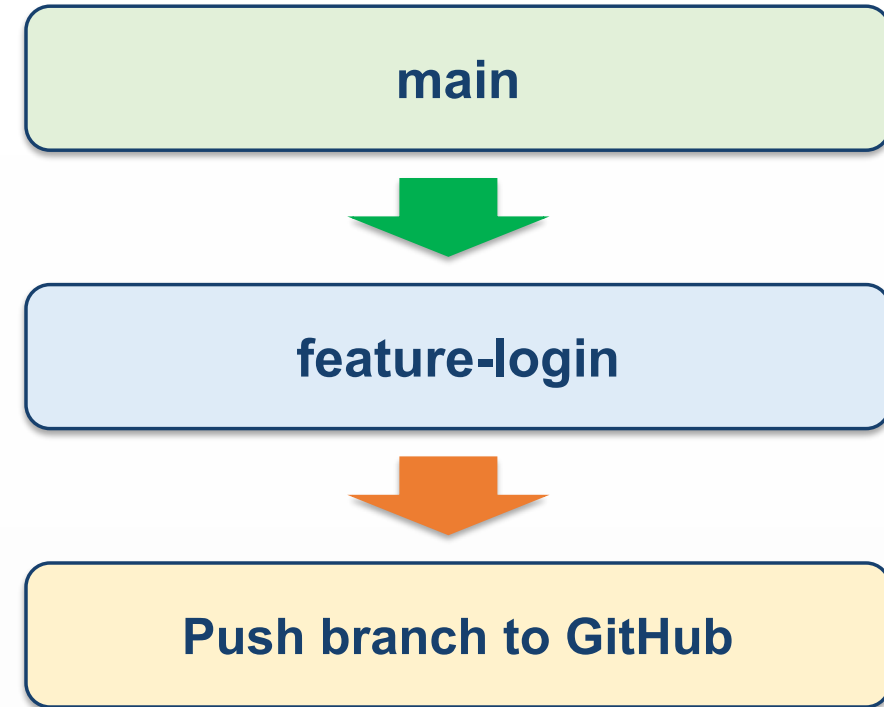
- Assignee
- Labels
- Milestone
- Comments



Feature Branch Workflow

Work on a branch — not directly on main

```
git switch main
git pull
git switch -c feature-login
# make a small change
git add .
git commit -m "Add login feature"
git push -u origin feature-login
```



Next step: open a Pull Request.

Pull Request (PR) Workflow



What is a Pull Request?

A request to merge changes from one branch into another. It creates a place for discussion, review and automated checks.

Common PR actions

- Add description
- Request reviewers
- Discuss changes
- Merge or close

Code Review & Merge

COMMENT

Ask a question or suggest an improvement.

APPROVE

The changes look good and can move forward.

REQUEST CHANGES

The author should update the PR before merge.

Simple team workflow

Developer A → opens PR Developer B → reviews Team → merges into main

Review before merge helps protect shared branches.

Remote Collaboration Conflict — Simple Example

Student A
message.txt → Hello from A
(pushes first)



Student B
message.txt → Hello from B
(local change)

Student B runs:

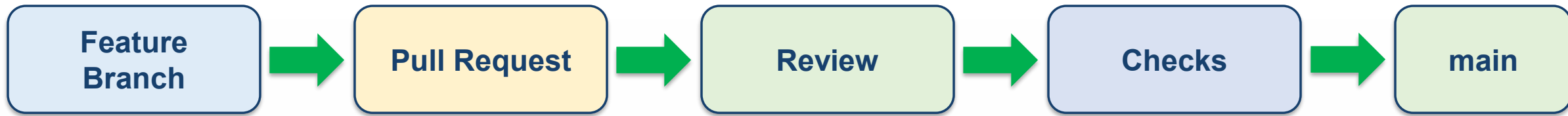
```
git pull
```

Git cannot choose the correct line

```
                <<<<<<< HEAD
Hello from B
=====
Hello from A
>>>>>> origin/main
```

Resolve the file → git add → git commit → git push

Protecting main — Branch Protection & Rulesets



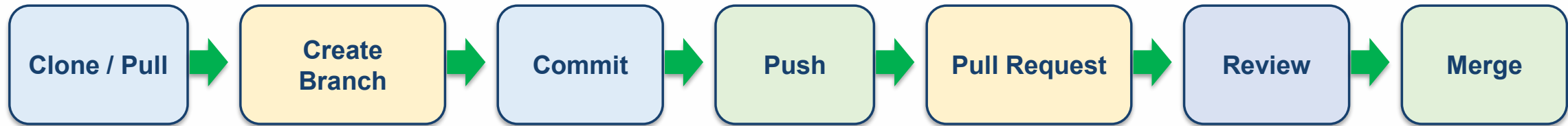
Typical protection rules

- Require a Pull Request
- Require approving reviews
- Require status checks to pass
- Block force pushes / deletions

Why DevOps teams use them

- Protect stable code
- Standardize collaboration
- Connect CI checks to merging
- Prepare for GitHub Actions on Day 3

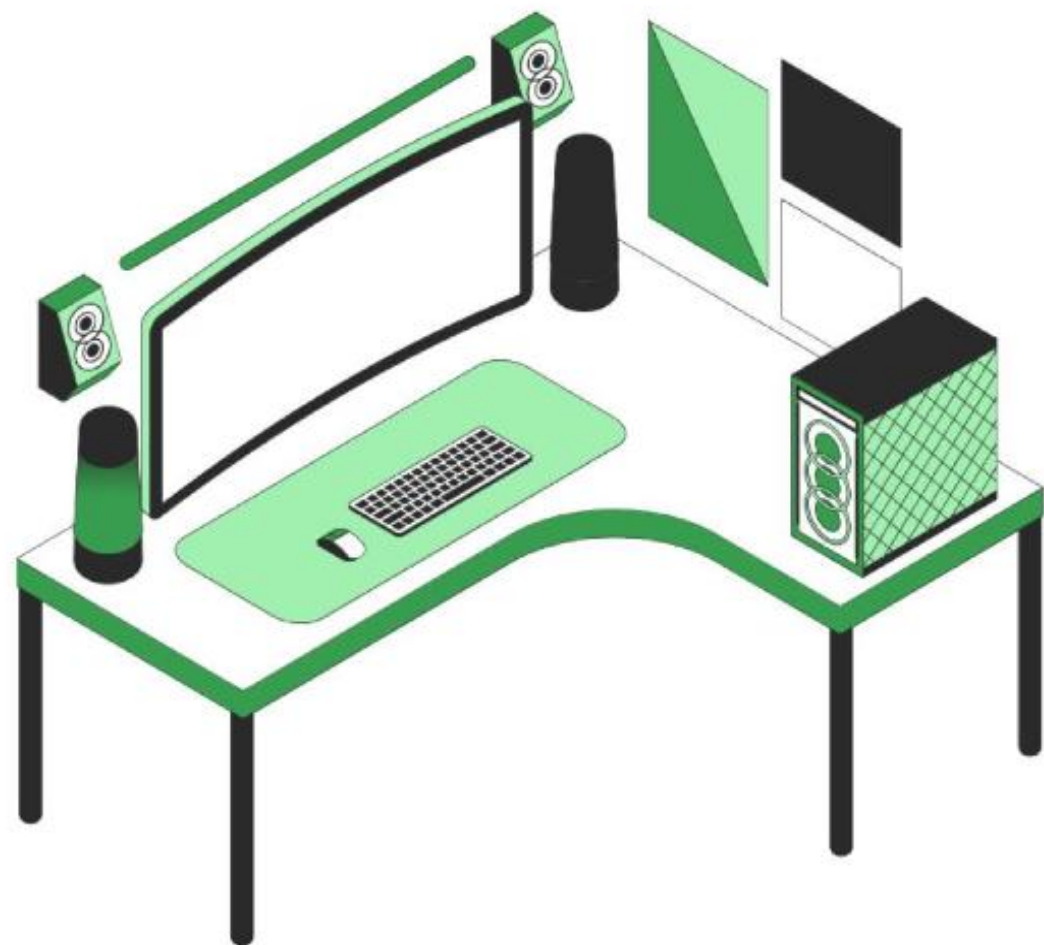
Day 2 — The Complete Workflow



Local Git + Remote GitHub + Team Collaboration

```
git clone      git pull      git switch -c feature  
git add .      git commit    git push      → Pull Request
```

Next: GitHub Actions and DevOps automation.



Do you have any questions?

Send it to us! We hope you learned something new.